

comunicacion-asincrona

- Clase 04 — Comunicación asíncrona
 - 1. Introducción
 - 2. Síncrono, asíncrono y estado del trabajo
 - 3. Modelo mental de mensajería distribuida
 - 4. Topologías de entrega y escalado
 - 5. Entrega, confirmaciones e idempotencia
 - 6. Publicación confiable: Transactional Outbox
 - 7. Reintentos, mensajes venenosos y DLQ
 - 8. Observabilidad y capacidad
 - 9. EventOS: compra confirmada e inferencia asíncrona
 - 10. Trade-offs y decisiones arquitectónicas
 - 11. Puntos clave
 - 12. Referencias y profundización

Clase 04 — Comunicación asíncrona

ARQ 2026 · UCC

1. Introducción

Una operación de negocio rara vez termina al responder una petición HTTP. Después de confirmar una compra puede ser necesario emitir una entrada, enviar una notificación, actualizar una métrica, generar un comprobante o ejecutar un análisis que toma varios segundos. Si una única solicitud debe esperar a que todos esos efectos terminen, el tiempo de respuesta y la disponibilidad de cada dependencia quedan acoplados a la experiencia principal.

La **comunicación asíncrona** permite separar la aceptación de un trabajo de su procesamiento posterior. Para hacerlo se intercambian mensajes a través de un broker: un productor publica un mensaje y uno o más consumidores lo procesan cuando tienen capacidad. Este modelo desacopla componentes, absorbe picos y permite escalar el trabajo, pero introduce consistencia eventual y fallos que no existen —o son menos visibles— dentro de una llamada local. Un mensaje puede duplicarse, demorarse o quedar sin procesar si no se diseña y opera el flujo correctamente.

El objetivo no es convertir todo en asíncrono. Es decidir qué efectos pueden terminar más tarde, definir las garantías necesarias y construir consumidores correctos cuando el mundo real entrega mensajes más de una vez.

2. Síncrono, asíncrono y estado del trabajo

En una interacción **síncrona**, quien invoca espera una respuesta para continuar. Una API que confirma un pago necesita, por ejemplo, saber antes de responder si la autorización fue aceptada. Esto ofrece una respuesta inmediata y una secuencia fácil de seguir, pero crea **acoplamiento temporal**: ambas partes deben estar disponibles al mismo tiempo y la latencia de cada salto se acumula.

En una interacción **asíncrona**, quien solicita o produce el trabajo puede continuar antes de que el efecto final termine. La asincronía no significa que el resultado sea irrelevante: significa que el resultado tiene un ciclo de vida separado. Conviene distinguir tres momentos:

1. **Aceptación**: el servicio validó que puede intentar el trabajo.
2. **Registro durable**: la intención o el hecho de negocio fue persistido.
3. **Finalización**: un consumidor completó el efecto derivado.

Por ejemplo, una plataforma puede confirmar una compra de forma síncrona y emitir el email de la entrada de forma asíncrona. No debería, en cambio, responder “compra confirmada” si todavía no validó el pago o no reservó el stock que el negocio exige confirmar de inmediato.

2.1 Decidir si un efecto admite asincronía

La pregunta útil no es “¿la mensajería escala?”, sino: **¿puede el usuario o el negocio esperar para conocer este resultado?** La tabla resume criterios habituales.

Criterio	Suele favorecer síncrono	Suele favorecer asíncrono
Respuesta	El resultado define la respuesta actual	Puede informarse o consultarse después
Duración	Trabajo breve y predecible	Trabajo largo o de duración variable
Dependencias	Pocas y críticas para la decisión	Efectos derivados e independientes
Volumen	Flujo estable y acotado	Picos o procesamiento masivo
Consistencia	Debe ser inmediata	Se tolera convergencia posterior
Fallo	Debe impedir la operación	Puede reintentarse o tratarse luego

Una notificación, una actualización analítica o una recomendación generada por IA suelen admitir demora. Una reserva de inventario, una autorización de pago o la validación de un permiso normalmente no, aunque cada dominio debe explicitar su propia regla.

2.2 202 Accepted y trabajos de larga duración

Cuando una API acepta trabajo pero no puede terminarlo dentro de la solicitud, puede responder **202 Accepted**. Esta respuesta comunica que la petición fue aceptada para procesamiento, no que el resultado final sea exitoso. Debe incluir una referencia durable, por ejemplo un `job_id`:

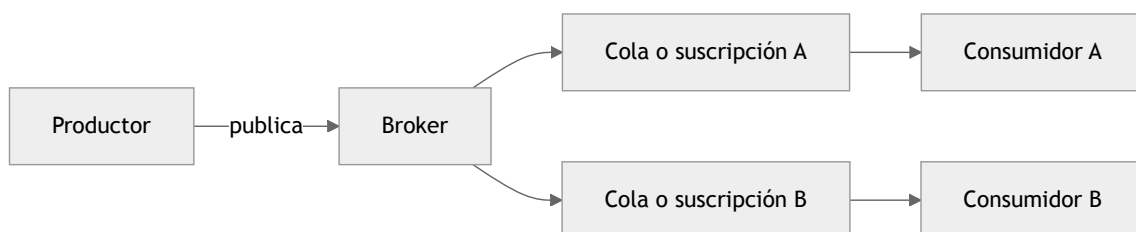
```
HTTP/1.1 202 Accepted
{
  "job_id": "inf-8f3c",
  "estado": "pendiente"
}
```

El sistema conserva el estado del trabajo (`pendiente`, `en_proceso`, `completado`, `fallido`) para que pueda consultarse o notificarse más tarde. Retornar **202** sin persistir el trabajo ni ofrecer una forma de conocer su desenlace solo traslada incertidumbre al usuario.

3. Modelo mental de mensajería distribuida

Un sistema de mensajería tiene cinco roles principales:

- **Productor**: crea y publica un mensaje.
- **Mensaje**: contrato autocontenido que describe un hecho, una orden o una tarea.
- **Broker**: recibe, persiste o enruta mensajes hacia destinos configurados.
- **Destino**: cola, suscripción o canal donde espera el mensaje.
- **Consumidor**: recibe y ejecuta el procesamiento asociado.



El broker no es la fuente de verdad del negocio. Una cola puede guardar un mensaje durante un tiempo, pero el estado de una compra, un usuario o una reserva sigue perteneciendo al servicio que modela ese dominio. Tampoco convierte la operación en confiable por sí solo: productor y consumidor deben decidir qué se confirma, qué se reintenta y cómo se detectan efectos duplicados.

3.1 Desacoples que aporta un broker

La mensajería reduce tres acoplamientos diferentes:

- **Espacial:** el productor no necesita conocer dirección, instancia ni cantidad de consumidores.
- **Temporal:** productor y consumidor no necesitan estar disponibles al mismo tiempo; el mensaje puede esperar.
- **De ritmo:** un productor puede generar trabajo más rápido que un consumidor durante un período limitado; la cola forma un *backlog*.

El último punto es útil, pero no mágico. Si se producen 10 000 mensajes por minuto y el sistema solo procesa 1 000, la cola crece indefinidamente. El backlog compra tiempo para escalar, limitar producción o degradar trabajo no crítico; no crea capacidad infinita.

3.2 El mensaje como contrato

Un mensaje debe ser comprensible sin que el consumidor lea la base de datos privada del productor. Como mínimo suele necesitar un tipo, una versión, un identificador único, momento de creación, clave de correlación y los datos necesarios para actuar.

```
{
  "id": "msg-52bd",
  "tipo": "compra.confirmada",
  "version": 1,
  "ocurrido_en": "2026-09-02T21:16:04Z",
  "correlation_id": "req-713a",
  "compra_id": "cmp-401",
  "usuario_id": "usr-88",
  "total": 24500
}
```

Un contrato pequeño y versionado reduce acoplamiento. Agregar campos opcionales suele ser compatible; cambiar el significado de un campo existente, eliminarlo o reutilizar un tipo para otro propósito puede romper consumidores que evolucionan a distinto ritmo.

3.3 Evento, comando y tarea

Los tres formatos se parecen técnicamente, pero expresan intenciones distintas:

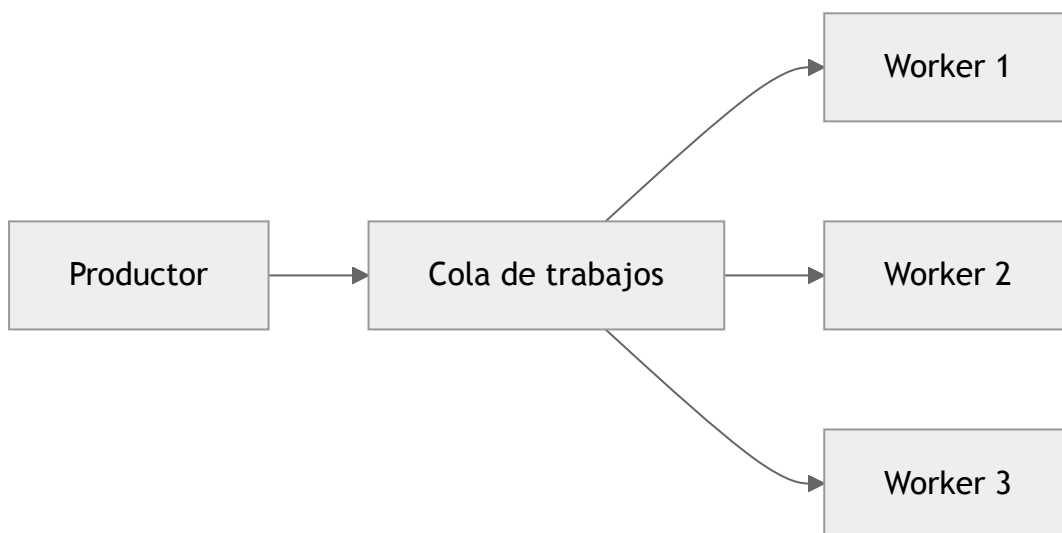
Forma	Significado	Destinatario típico	Ejemplo
Evento	Un hecho ya ocurrió	Cero o varios interesados	compra.confirmada
Comando	Se solicita una acción	Un responsable lógico	emitir-entrada
Tarea	Unidad de trabajo ejecutable	Un worker	generar-resumen-ia

Un evento no ordena que alguien haga algo: comunica un hecho pasado. Si un productor publica `compra.confirmada`, puede sumarse un nuevo consumidor sin modificarlo. Un comando tiene un destinatario y una expectativa de acción más explícita. Mezclarlos vuelve opaco quién es dueño de una decisión de negocio.

4. Topologías de entrega y escalado

4.1 Cola de trabajo y consumidores competidores

En una **cola de trabajo**, cada mensaje debe ser procesado por un único consumidor lógico. Si hay varias instancias del mismo worker, compiten por los mensajes para repartir carga.



Es adecuada para enviar emails, generar archivos, transformar imágenes o ejecutar inferencias. Agregar workers aumenta el paralelismo si el trabajo no requiere una secuencia global. La operación debe ser idempotente: “un consumidor lógico” no significa que el broker nunca reentregará el mensaje después de un fallo.

Una cola puede tener políticas distintas de extracción:

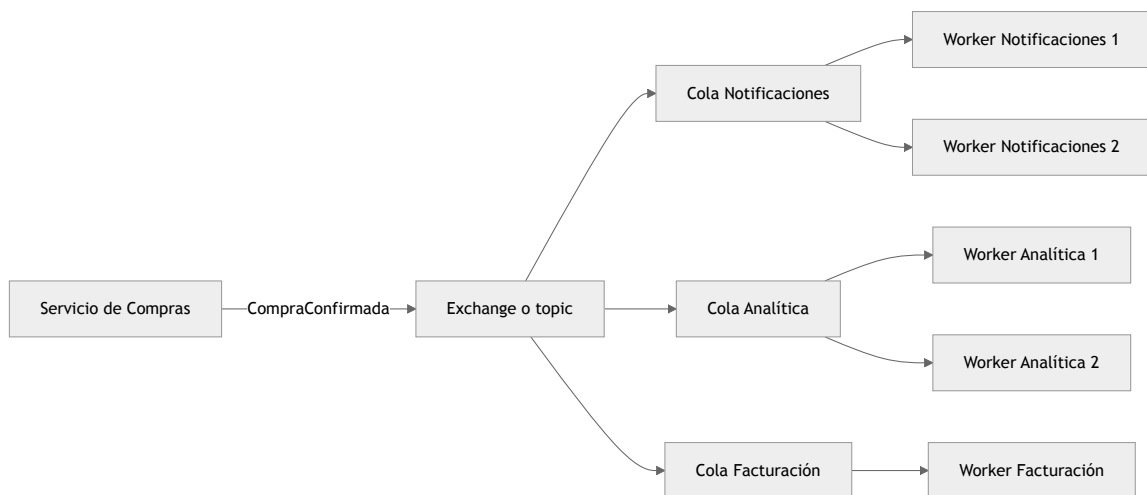
- **FIFO:** intenta respetar el orden de llegada, sujeto a reintentos y paralelismo. Si un consumidor rechaza un mensaje y solicita su *requeue*, el mensaje vuelve a estar disponible para una nueva entrega; junto con varios consumidores y mensajes en vuelo, la secuencia de procesamiento observada puede diferir de la secuencia de publicación. FIFO no equivale a orden global de efectos.
- **Prioridad:** permite adelantar trabajo crítico, por ejemplo una alerta de fraude frente a un email de bienvenida. Puede postergar indefinidamente el trabajo de menor prioridad si no se controla.
- **Diferida:** conserva una tarea hasta un momento futuro, útil para recordatorios o reintentos con demora.

Estas políticas cambian qué mensaje sale primero; no cambian el modelo de consumidores competidores ni eliminan la necesidad de capacidad suficiente.

4.2 Publish/Subscribe

En **Publish/Subscribe (Pub/Sub)**, un evento se entrega a múltiples suscripciones independientes. Cada una mantiene su propia velocidad, reintentos y política de fallos.

En una implementación real, Pub/Sub y consumidores competidores se **componen**. El broker copia `CompraConfirmada` en una cola por cada suscriptor lógico; luego, varias instancias del mismo servicio compiten por los mensajes de su cola. Así, Notificaciones puede escalar con varios workers sin que una instancia de Analítica reciba trabajo de Notificaciones.



El beneficio principal es que Compras no necesita conocer a cada destino. Agregar Analítica no obliga a redeplegar Compras. El costo es que los efectos quedan distribuidos: hay que observar qué consumidores existen, quién es responsable de cada uno y si todos procesan a tiempo.

Pub/Sub también puede propagar invalidaciones. Cuando cambia el detalle público de un evento, el servicio dueño puede publicar `evento.actualizado`; las instancias que mantienen una copia de caché invalidan su entrada. El servicio de Eventos no necesita saber cuántas cachés existen ni dónde están. Esto no vuelve automáticamente correcta la caché: el consumidor debe tolerar reintentos y la aplicación debe seguir definiendo el TTL y la fuente de verdad.

4.3 Paralelismo, partición y orden

El paralelismo introduce una tensión: procesar más rápido suele reducir la garantía de orden global. Incluso si el broker entrega `A` antes que `B`, dos workers pueden terminar en el orden inverso. Por eso hay que separar:

- **Orden de entrega:** el broker entrega mensajes en cierta secuencia.
- **Orden de procesamiento:** los consumidores empiezan a ejecutarlos en cierta secuencia.
- **Orden de efectos:** las escrituras o llamadas externas terminan en cierta secuencia.

Cuando importa el orden de un agregado, se usa una **clave de partición lógica**. Todas las actualizaciones de una compra, por ejemplo, se encaminan a la misma partición o secuencia; compras distintas pueden procesarse en paralelo. No conviene usar un único canal global “para preservar orden”, porque convierte el sistema entero en una fila y limita la capacidad.

Cuando incluso el orden por clave no puede garantizarse por completo, el consumidor debe comparar versiones, rechazar transiciones inválidas o modelar operaciones conmutativas. Diseñar para tolerar mensajes tardíos suele escalar mejor que exigir orden total.

4.4 RabbitMQ y otras familias de brokers

RabbitMQ es un broker orientado a colas y enrutamiento. Un productor publica en un **exchange**; las reglas llamadas **bindings** llevan el mensaje a una o más **queues**; los consumidores reciben desde esas colas.

Exchange de RabbitMQ	Idea conceptual	Uso típico
direct	Enrutamiento por clave exacta	Separar tipos de tarea
fanout	Copiar a todos los destinos enlazados	Difundir un evento
topic	Enrutamiento por patrón de clave	Suscribirse a familias como <code>compra.*</code>

RabbitMQ suele **empujar** mensajes hacia consumidores con capacidad, limitada por el *prefetch*: cantidad de mensajes no confirmados que una instancia puede tener en vuelo. Un valor demasiado alto puede concentrar mensajes en un worker lento; `0` significa sin límite y debe usarse con cuidado, porque un consumidor lento puede acumular un búfer de mensajes sin confirmar y agotar memoria. Un *prefetch* de `1` favorece un reparto más equitativo (*fair dispatch*) entre workers, pero reduce el *throughput*, especialmente si hay latencia entre el ACK y la siguiente entrega. El valor adecuado depende de la duración, variabilidad y costo de memoria del trabajo.

Kafka representa otra familia: conserva eventos en un log retenido y el consumidor los obtiene con *polling*, avanzando su propia posición. Esto facilita ciertos escenarios de retención y reprocesamiento, pero no implica que sea universalmente mejor. La elección depende del patrón de entrega, la necesidad de enrutamiento, retención, operación y experiencia del equipo.

5. Entrega, confirmaciones e idempotencia

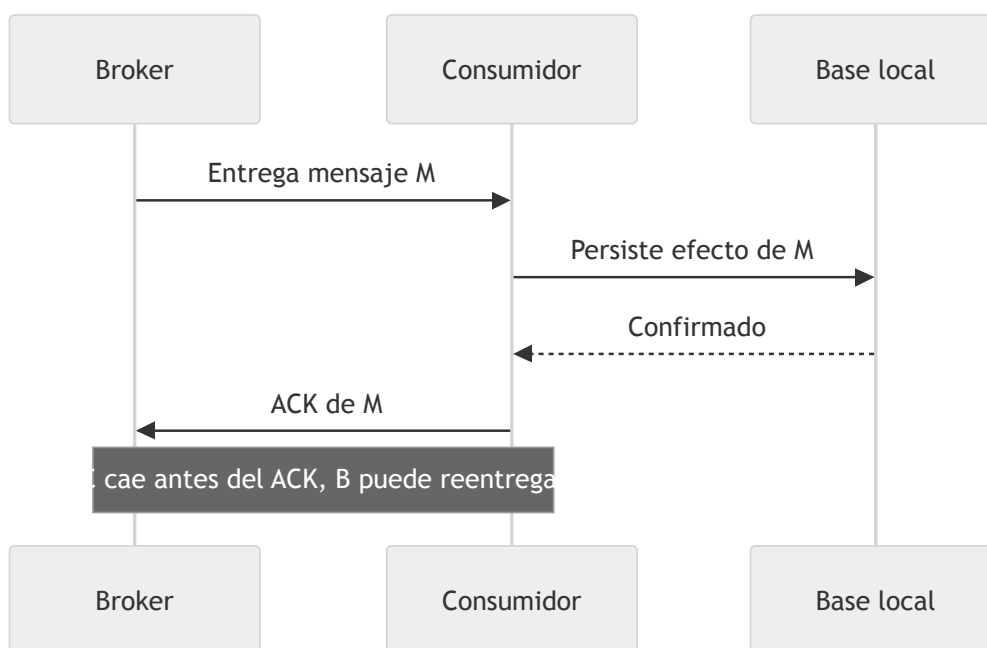
Los sistemas distribuidos no pueden evitar por completo la incertidumbre ante un fallo de red: si un consumidor procesó un mensaje y se cayó antes de confirmar, el broker no puede saber con certeza si el efecto ocurrió. De allí surgen semánticas de entrega.

Semántica	Riesgo principal	Consecuencia
Como máximo una vez	Perder mensajes	No hay reentrega tras un fallo
Al menos una vez	Duplicar mensajes	Se reintenta hasta confirmar procesamiento
Exactamente una vez	Difícil de garantizar de extremo a extremo	Requiere límites precisos; no se obtiene solo activando una opción del broker

La alternativa más frecuente es **al menos una vez**. Prioriza no perder un trabajo durable, a cambio de exigir que los consumidores toleren duplicados.

5.1 Confirmar después del efecto durable

Un consumidor recibe un mensaje, produce un efecto y luego lo confirma. Si confirma primero y se cae antes de guardar el efecto, hay pérdida lógica: el broker no lo volverá a entregar. Si guarda el efecto y se cae antes de confirmar, habrá reentrega. Esta segunda situación es preferible solo si el efecto es idempotente.



Un ACK confirma que el consumidor terminó su responsabilidad durable, no que “empezó a trabajar”. La política concreta depende del broker, pero el principio es independiente de la tecnología.

5.2 Idempotencia

Un procesamiento es **idempotente** si aplicar el mismo mensaje varias veces deja el mismo estado final que aplicarlo una sola vez. La Clase 02 ya presentó la estrategia base al tratar el dual write: registrar el `mensaje_id` procesado junto con el efecto de negocio, en la misma transacción local. Acá interesa ampliar el repertorio para los casos en que ese registro no alcanza o no está disponible:

- **Restricción única**, por ejemplo `entrada.compra_id`, para impedir una segunda creación sin necesitar una tabla de mensajes procesados.
- **Transición de máquina de estados**: modelar los estados válidos del agregado y aceptar una transición solo desde el estado esperado. Por ejemplo, `pendiente → emitida` se aplica una vez; si llega otra vez el mismo mensaje cuando la entrada ya está `emitida`, o llega una transición incompatible cuando está `cancelada`, el consumidor no repite el efecto ni retrocede el estado.
- **Clave idempotente** enviada a una API externa que la soporte, conservando la relación con el mensaje original.

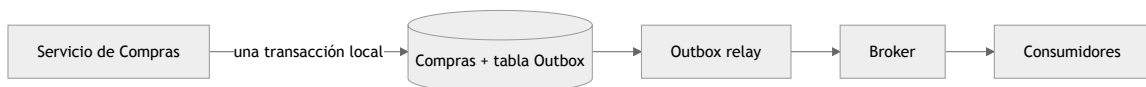
Cualquiera sea la estrategia, el registro de duplicados debe confirmarse junto con el efecto, en la misma transacción: marcar un mensaje como procesado antes de completar el efecto produce pérdida lógica si el proceso falla inmediatamente después.

5.3 Efectos externos

La transacción local no abarca un proveedor de email, pagos o una API de terceros. Por eso la idempotencia debe atravesar el límite externo: usar una clave idempotente ofrecida por el proveedor, registrar el intento y consultar su resultado antes de repetir, o diseñar el destinatario para deduplicar. “Reintentar” sin esta decisión puede cobrar dos veces o enviar comunicaciones duplicadas.

6. Publicación confiable: Transactional Outbox

La Clase 02 introdujo el problema del **dual write** —guardar la compra y publicar `CompraConfirmada` son dos operaciones sobre recursos distintos que pueden fallar por separado— y el patrón **Transactional Outbox** como solución: persistir la entidad de negocio y un registro de evento pendiente en la misma transacción local, para que un proceso posterior —el *relay*— los publique al broker.



Lo que agrega esta clase es el ciclo de vida operativo del registro:

1. La transacción guarda la compra y el evento con un identificador estable.
2. Si la transacción falla, no existe ni compra ni evento pendiente.
3. Si confirma, el relay reintentará publicar hasta obtener confirmación del broker.
4. Luego marca el registro como publicado o conserva información suficiente para auditarlo.

El relay también puede fallar después de publicar y antes de marcar el registro. Al reintentarlo, el broker recibe el evento otra vez. Por eso Outbox evita perder la intención de publicar, pero normalmente mantiene una entrega al menos una vez: los consumidores deben seguir siendo idempotentes.

6.1 Cómo obtiene eventos el relay

El relay necesita detectar registros nuevos de la tabla Outbox. Las dos estrategias clásicas son:

- **Polling Publisher:** un proceso consulta periódicamente los registros pendientes, los publica y actualiza su estado. Es simple de implementar y funciona con una base de datos habitual; a cambio, incorpora consultas repetidas, una demora de sondeo y requiere coordinar varios relays para que no tomen el mismo registro sin control.
- **Change Data Capture (CDC):** un conector lee el log de transacciones de la base de datos y detecta las inserciones en la tabla Outbox. Luego las transforma o enruta al broker. Reduce el sondeo de la tabla y puede capturar los cambios en el orden del log, pero incorpora infraestructura, permisos y operación del conector. Debezium es un ejemplo conocido de esta alternativa.

Ambas estrategias mantienen la misma garantía fundamental: el cambio de negocio y el registro Outbox se confirman juntos. Ninguna elimina por sí sola la posibilidad de publicar dos veces, por lo que los consumidores deben seguir siendo idempotentes.

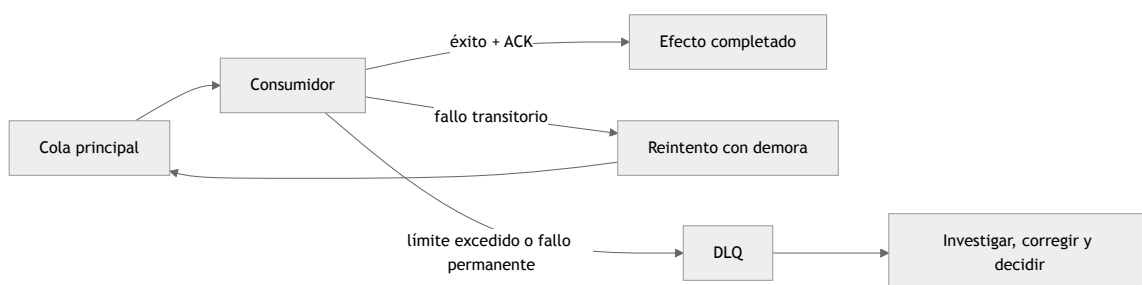
El patrón agrega costos reales: tabla adicional, proceso de relay, limpieza o archivado, monitoreo de registros atrasados y consistencia eventual entre la compra y sus efectos derivados. No reemplaza una transacción distribuida ni vuelve atómica una decisión que necesita coordinar varios dominios.

7. Reintentos, mensajes venenosos y DLQ

No todos los fallos deben tratarse igual. Un timeout de una dependencia, una saturación temporal o una red inestable pueden recuperarse. Un payload inválido, una versión de contrato incompatible o una regla de negocio imposible de satisfacer rara vez mejoran al reintentar inmediatamente.

Un reintento debe tener una política explícita: cantidad máxima, demora, *backoff* y motivo registrado. El **backoff exponencial** incrementa la espera entre intentos; agregar *jitter* evita que muchos consumidores reintenten al mismo tiempo después de una caída. Reintentar infinitamente y sin demora amplifica una dependencia caída y consume capacidad útil.

Un **mensaje venenoso** es aquel que falla de modo repetible. Puede bloquear una cola, llenar logs y ocultar fallos nuevos. Una **Dead Letter Queue (DLQ)** aísla mensajes que excedieron la política definida para que puedan analizarse sin detener el resto del flujo.



Una DLQ no es una papelera. Debe tener dueño, alerta, retención definida y procedimiento para investigar. Reinyectar un mensaje sin corregir la causa solo repite el incidente; descartarlo sin decisión puede perder un efecto de negocio. A veces se corrige el consumidor y se reprocesa; otras, se corrige el dato y se produce un nuevo mensaje válido.

Estos mecanismos resuelven fallos dentro de un flujo de mensajería. Circuit breaker, bulkhead y otras políticas de resiliencia transversal frente a fallos de red o de dependencias se desarrollan en la Clase 08.

8. Observabilidad y capacidad

Una petición síncrona suele ofrecer su latencia en la respuesta. En un flujo asíncrono se necesita observar el recorrido completo. Un `correlation_id` propagado desde la solicitud inicial permite unir logs y eventos de productor, relay y consumidores.

Las métricas mínimas son:

- Profundidad de la cola y antigüedad del mensaje más viejo.
- Tasa de publicación y de consumo.
- Tiempo desde la publicación hasta la finalización del efecto.
- Mensajes en vuelo, reintentos y errores por tipo.
- Cantidad y antigüedad de mensajes en DLQ.

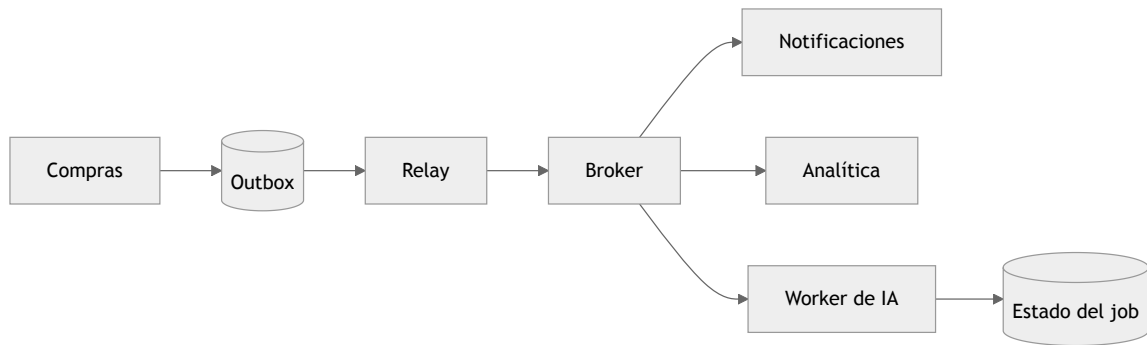
El **consumer lag** expresa el trabajo producido que todavía no fue procesado. Un lag creciente puede deberse a un pico legítimo, consumidores detenidos, un cambio que los volvió lentos o una dependencia externa degradada.

Responder a ese crecimiento es una decisión de **backpressure**: agregar consumidores cuando el trabajo es paralelizable, limitar la tasa de producción o degradar funcionalidades no críticas antes de que el backlog se vuelva inmanejable. La respuesta no es siempre “agregar instancias”: si el recurso limitante es una API externa, escalar consumidores puede agravar la saturación en lugar de resolverla.

Los objetivos de servicio también deben expresar el resultado de negocio: por ejemplo, “el 99 % de las entradas se emite en menos de dos minutos desde la compra confirmada”, no solo “el broker aceptó el mensaje”. Esa diferencia revela si el sistema está cumpliendo el efecto que importa.

9. EventOS: compra confirmada e inferencia asíncrona

EventOS confirma una compra dentro de su propio servicio y registra `CompraConfirmada` en un Outbox. El relay publica el evento. Notificaciones emite la entrada, Analítica registra la conversión y un servicio de IA puede clasificar señales para soporte o fraude no bloqueante. Cada consumidor tiene su propia cola o suscripción, por lo que la demora de una inferencia no retrasa el email ni invalida una compra ya confirmada.



Durante un pico de ventas, el sistema puede priorizar efectos críticos y observar la antigüedad de las colas. Si el worker de IA se atrasa, se escala si el modelo y el proveedor lo permiten, o se degrada esa funcionalidad no esencial. El diseño no asume que cada evento llega una sola vez: Notificaciones debe evitar una entrada duplicada y Analítica debe evitar doble conteo.

10. Trade-offs y decisiones arquitectónicas

Decisión	Beneficio	Costo o riesgo	Conviene cuando
Síncrono	Resultado inmediato y flujo simple	Acoplamiento temporal y latencia acumulada	El resultado decide la operación actual
Asíncrono	Desacople, absorción de picos y escalado	Estado eventual y operación adicional	El efecto puede terminar después
Cola de trabajo	Reparte unidades de trabajo	Un único efecto lógico; requiere idempotencia	Hay workers equivalentes
Pub/Sub	Agrega consumidores sin cambiar al productor	Más efectos y ownership distribuido	Un hecho interesa a varios dominios
Al menos una vez	Minimiza pérdida de trabajo	Duplicados y reentregas	Se pueden diseñar consumidores idempotentes
Outbox	Evita perder eventos tras una transacción local	Relay, tabla y consistencia eventual	Un cambio durable debe publicar un hecho
DLQ	Aísla mensajes problemáticos	Requiere operación y proceso de recuperación	Los reintentos no siempre resuelven fallos

No hay una garantía gratis: durabilidad, orden, baja latencia, paralelismo y simplicidad tensionan entre sí. La arquitectura correcta explicita qué efecto es crítico, qué demora admite y quién responde cuando el flujo no concluye normalmente.

11. Puntos clave

- La asincronía separa la aceptación de un trabajo de su finalización; no elimina la responsabilidad de informar y persistir su estado.
 - Usar mensajería tiene sentido cuando el efecto puede completarse después, absorber picos o desacoplar consumidores; no para evadir decisiones que requieren resultado inmediato.
 - Una cola de trabajo entrega cada mensaje a un consumidor lógico; Pub/Sub entrega un evento a varios interesados independientes.
 - El paralelismo no garantiza orden global: si importa, hay que particionar por clave y diseñar para mensajes tardíos.
 - La entrega al menos una vez reduce pérdida de mensajes, pero exige consumidores idempotentes.
 - Un ACK debe emitirse después de completar el efecto durable; confirmar antes puede producir pérdida lógica.
 - Transactional Outbox confirma el cambio de negocio y la intención de publicar dentro de la misma transacción local, pero no elimina duplicados.
 - Reintentos necesitan límites, demora y clasificación de fallos; los mensajes venenosos se aíslan en una DLQ con proceso operativo.
 - Medir lag, antigüedad, tiempo hasta el efecto y DLQ permite saber si el flujo cumple su objetivo de negocio.
-

12. Referencias y profundización

- [RabbitMQ Tutorials](#) — ejemplos oficiales de colas de trabajo, Pub/Sub y enrutamiento.
- [RabbitMQ — Consumer Prefetch](#) — límite de mensajes no confirmados y su efecto sobre el reparto y la capacidad.
- [Enterprise Integration Patterns — Messaging](#) — catálogo de patrones de integración por mensajería.
- [Microservices.io — Transactional Outbox](#) — motivación, garantías y alternativas del patrón.
- [Debezium — Outbox Event Router](#) — ejemplo de CDC para capturar cambios en una tabla Outbox.
- [Designing Data-Intensive Applications](#) — referencia sobre logs, mensajería, consistencia y sistemas distribuidos.